Mybatis讲义

mybatis应用

- 连接数据库，在db.properties文件中添加连接信息

```
jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://192.168.242.101:3306/mybatis?characterEncoding=UTF-8&rewriteBatchedStatements=true
jdbc.username=root
jdbc.password=123456
```

- 在mybatis的配置文件mybatis-config.xml中引入该文件

```
    <properties resource="db.properties"></properties>
```

- 然后把数据库连接信息保存到environments标签里面

```
<environments default="development">
  <environment id="development">
    <transactionManager type="JDBC"/><!-- 单独使用时配置成MANAGED没有事务 -->
    <dataSource type="POOLED" >
      <property name="driver" value="${jdbc.driver}"/>
      <property name="url" value="${jdbc.url}"/>
      <property name="username" value="${jdbc.username}"/>
      <property name="password" value="${jdbc.password}"/>
    </dataSource>
  </environment>
</environments>
```

- 可以配置全局的settings信息

```
<settings>
  <!-- 打印查询语句 -->
  <setting name="logImpl" value="STDOUT_LOGGING" />

  <!-- 控制全局缓存（二级缓存），默认 true-->
  <setting name="cacheEnabled" value="true"/>

  <!-- 延迟加载的全局开关。当开启时，所有关联对象都会延迟加载。默认 false  -->
  <setting name="lazyLoadingEnabled" value="true"/>


  <!-- 当开启时，任何方法的调用都会加载该对象的所有属性。默认 false，可通过select标签的 fetchType
```

```
来覆盖-->
  <setting name="aggressiveLazyLoading" value="true"/>
  <!--  Mybatis 创建具有延迟加载能力的对象所用到的代理工具，默认JAVASSIST -->
  <!--<setting name="proxyFactory" value="CGLIB" />-->
  <!-- STATEMENT级别的缓存，使一级缓存，只针对当前执行的这一statement有效 -->

  <setting name="localCacheScope" value="STATEMENT"/>

  <setting name="localCacheScope" value="SESSION"/>
</settings>
```

- 配置别名：这个是用来将来sql语句接收的时候定义的，如果不定义则就用全类名接收是一样的

```
<typeAliases>
  <typeAlias alias="blog" type="com.gupaoedu.domain.Blog" />
</typeAliases>
```

- 然后就开始配置写sql语句的xml文件，有两种方式

```
<mappers>
  <mapper resource="BlogMapper.xml"/>
  <mapper resource="BlogMapperExt.xml"/>
  <!-- 这个需要把xml文件放入到指定的包名下 -->
  <!--<package name="com.gupaoedu.mapper"/>-->
</mappers>
```

- 开始写实体类，在domain包下定义实体类，比如：

```
public class Blog implements Serializable{
    Integer bid; // 文章ID
    String name; // 文章标题
    @DemoAnnotion
    Integer authorId; // 文章作者ID

    public Integer getBid() {
        return bid;
    }

    public void setBid(Integer bid) {
        this.bid = bid;
    }

    public String getName() {
        return name;
    }


    public void setName(String name) {
```

```
        this.name = name;
    }

    public Integer getAuthorId() {
        return authorId;
    }

    public void setAuthorId(Integer authorId) {
        this.authorId = authorId;
    }

    @Override
    public String toString() {
        return "Blog{" +
                "bid=" + bid +
                ", name='" + name + '\'' +
                ", authorId='" + authorId + '\'' +
                '}';
    }
}
```

- 然后在mapper包下定义mapper接口类,并在里面定义方法，比如：

```
public interface BlogMapper {
    /**
     * 根据主键查询文章
     * @param bid
     * @return
     */
    public Blog selectBlogById(Integer bid);
}
```

- 然后定义sql文件，比如：可以在里面二级缓存，使用"标签，也可以使用redis做缓存，然后把sql语句写在该文件中，文件中的namespace就是刚才定义的mapper类的全路径，这个必须唯一，然后里面的sql的id可以不唯一，因为底层使用的namespace+sqlId保证唯一就可以了

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.gupaoedu.mapper.BlogMapper" >
    <!-- 声明这个namespace使用二级缓存 -->
<!--    <cache/>-->

    <!-- 使用Redis作为二级缓存 -->
<!--    <cache type="org.mybatis.caches.redis.RedisCache"-->
<!--           eviction="FIFO" flushInterval="60000" size="512" readOnly="true"/>-->

        <cache type="org.apache.ibatis.cache.impl.PerpetualCache"

               size="1024"
```

```
                eviction="LRU"
                flushInterval="120000"
                readOnly="false"/>

    <!-- 根据文章查询评论，一对多 -->
    <select id="selectBlogWithCommentById" resultMap="BlogWithCommentMap" >
        select b.bid, b.name, b.author_id authorId, c.comment_id commentId, c.content
        from blog b, comment c
        where b.bid = c.bid
        and b.bid = #{bid}
    </select>
</mapper>
```

到此，mybatis的开发就写完了

Mybatis源码分析

初始化

- 首先加载配置文件，并且解析配置文件，把每个sql都封装成一个statement保存起来

```
@Before
public void prepare() throws IOException {
  String resource = "mybatis-config.xml";
  InputStream inputStream = Resources.getResourceAsStream(resource);
  sqlSessionFactory = new SqlSessionFactoryBuilder().build(inputStream);
}
```

- 主要查看build()方法

```
// 1.首先解析配置文件
// 用于解析 mybatis-config.xml，同时创建了 Configuration 对象 ，所有xml中的元素都可以在configuration中获得
XMLConfigBuilder parser = new XMLConfigBuilder(inputStream, environment, properties);

// 2.把文件中的标签都封装成一个dom文件
public XPathParser(InputStream inputStream, boolean validation, Properties variables, EntityResolver entityResolver) {
  commonConstructor(validation, variables, entityResolver);
  this.document = createDocument(new InputSource(inputStream));
}

// 3.开始对每一个标签进行解析，从configuration标签开始
public Configuration parse() {
  if (parsed) {
    throw new BuilderException("Each XMLConfigBuilder can only be used once.");
  }
  parsed = true;
```

```
  // XPathParser, dom 和 SAX 都有用到 ->点进去,解析配置文件把数据放入到configuration中
  parseConfiguration(parser.evalNode("/configuration"));
  return configuration;
}

// 4.以下是所有标签的解析步骤，都会保存到configuration类中去，后面那信息都去这个里面去拿
// 对于全局配置文件各种标签的解析，这个properties标签节点会封装成一个Properties
propertiesElement(root.evalNode("properties"));
// 解析 settings 标签
// 1.获取settings节点下的所有配置的标签
// 2.判断配置的标签是否合法
Properties settings = settingsAsProperties(root.evalNode("settings"));
// 是否配置虚拟文件，通过该配置可以加载自定义的虚拟文件系统应用程序
loadCustomVfs(settings);
// 打印查询语句的日志logImpl
loadCustomLogImpl(settings);
// 类型别名，有两种方式，一种是package标签,一种是typeAlias标签,
// 最终都会保存到TypeAliasRegistry类中的typeAliases的map中,key为别名，值为类名
typeAliasesElement(root.evalNode("typeAliases"));
// 插件
// 1.先拿到配置的插件的类，然后通过构造实例化该类
// 2.看是否设置了其他属性，如果设置了把属性对象放入到interceptor对象中的Properties中
// 3.把类放入到InterceptorChain类中的list中保存起来
pluginElement(root.evalNode("plugins"));
// 用于创建对象，这个可以去定义自己的对象工厂，可以自己写个类继承DefaultObjectFactory,重写create
方法，然后在里面做逻辑判断
objectFactoryElement(root.evalNode("objectFactory"));
// 用于对对象进行加工
objectWrapperFactoryElement(root.evalNode("objectWrapperFactory"));
// 反射工具箱，可以获取Class 对象的构造器、属性及其的 getter 和 setter 方法
// 可以自定义MyReflectorFactory继承默认的DefaultReflectorFactory
reflectorFactoryElement(root.evalNode("reflectorFactory"));
// settings 子标签赋值，默认值就是在这里提供的
settingsElement(settings);
// 创建了数据源，把事务的工厂，数据源的配置信息都保存到configuration类中
environmentsElement(root.evalNode("environments"));
// 多数据源，<databaseIdProvider>要配置在<environments>的后面
// 在sql语句的标签添加databaseId="mysql"
databaseIdProviderElement(root.evalNode("databaseIdProvider"));
// 1.定义转换类继承类BaseTypeHandler
// 2.覆盖4个未实现的方法，其中setNonNullParameter为java程序设置数据到数据库的回调方法，
getNullableResult 为查询时 mysql的字符串类型转换成 java的Type类型的方法
// 3.在MyBatis核心配置文件中进行注册
typeHandlerElement(root.evalNode("typeHandlers"));
// 解析引用的Mapper映射器
mapperElement(root.evalNode("mappers"));
```

- 重点分析该方法mapperElement(root.evalNode("mappers"));

```
private void mapperElement(XNode parent) throws Exception {

  if (parent != null) {
```

```
    for (XNode child : parent.getChildren()) {
      // 不同的定义方式的扫描，最终都是调用 addMapper()方法（添加到 MapperRegistry）。这个方法和 getMapper() 对应
      // package:第一种配置方式
      if ("package".equals(child.getName())) {
        String mapperPackage = child.getStringAttribute("name");
        // 这个地方本质还是调用了MapperRegistry.addMapper方法，把包下面的都绑定起来，最后生成statement放入configuration
        configuration.addMappers(mapperPackage);
      } else {
        String resource = child.getStringAttribute("resource");
        String url = child.getStringAttribute("url");
        String mapperClass = child.getStringAttribute("class");
        if (resource != null && url == null && mapperClass == null) {
          // resource   相对路径
          ErrorContext.instance().resource(resource);
          InputStream inputStream = Resources.getResourceAsStream(resource);
          XMLMapperBuilder mapperParser = new XMLMapperBuilder(inputStream, configuration, resource, configuration.getSqlFragments());
          // 解析 Mapper.xml，总体上做了两件事情 ，把sql都封装成一个个的statement,然后我们再去把namespace+id绑定一个代理的工厂，后面用来调用
          // 他最终也是调用了MapperRegistry.addMapper
          mapperParser.parse();
        } else if (resource == null && url != null && mapperClass == null) {
          // url    绝对路径
          ErrorContext.instance().resource(url);
          InputStream inputStream = Resources.getUrlAsStream(url);
          XMLMapperBuilder mapperParser = new XMLMapperBuilder(inputStream, configuration, url, configuration.getSqlFragments());
          // 他最终也是调用了MapperRegistry.addMapper
          mapperParser.parse();
        } else if (resource == null && url == null && mapperClass != null) {
          // class  单个接口
          Class<?> mapperInterface = Resources.classForName(mapperClass);
          // 由于只有一个，则直接调用了MapperRegistry.addMapper
          configuration.addMapper(mapperInterface);
        } else {
          throw new BuilderException("A mapper element may only specify a url, resource or class, but not more than one.");
        }
      }
    }
  }
}
```

- 分析parse()方法

```
public void parse() {
    // 总体上做了两件事情，对于语句的注册和接口的注册
    if (!configuration.isResourceLoaded(resource)) {
```

```
      // 1、具体增删改查标签的解析。
      // 一个标签一个MappedStatement。 >>，由于有些是配置了mapper里面的文件名，则通过解析mapper标签
      configurationElement(parser.evalNode("/mapper"));
      configuration.addLoadedResource(resource);
      // 2、把namespace（接口类型）和工厂类绑定起来，放到一个map。
      // 一个namespace 一个 MapperProxyFactory
      bindMapperForNamespace();
    }
    // 还没有解析完成再此处去解析，如果感兴趣，具体含义可以自己去查下
    parsePendingResultMaps();
    parsePendingCacheRefs();
    parsePendingStatements();
  }
```

- configurationElement方法

```
private void configurationElement(XNode context) {
  try {
    // 获取配置信息的namespace
    String namespace = context.getStringAttribute("namespace");
    if (namespace == null || namespace.equals("")) {
      throw new BuilderException("Mapper's namespace cannot be empty");
    }
    // 设置当前的namespace
    builderAssistant.setCurrentNamespace(namespace);
    // 添加缓存对象
    //首先明确一点 cache-ref 只对二级缓存有效，没有使用二级缓存时，这东西没有意义。以下说明只针对二级缓存。
    // 当mybatis的当前命名空间存在DML的事务提交时，会使当前命名空间里的缓存失效，这样在查询时，会直接从数据库拿到数据，并再次缓存。
    // 但是如果是多表连接查询，如tableA join tableB，A表的DML操作在A的nameSpace,B表的DML操作在B的nameSpace，当B表在进行了数据修改时
    // 不会使A表缓存失效，再使用tableA join tableB会直接从缓存中获取数据，因为此时缓存没有刷新，而且B表的数据也有变化，那么此时读取的就是脏数据。
    // 此时就需要在B的nameSpace里使用<cache-ref namespace="A"/>,
    // 保证跟新B时，A的缓存也失效。再在A里面执行 tableA join tableB时，会重新从数据库里查询最新数据
    cacheRefElement(context.evalNode("cache-ref"));
    // 对解析 cache 属性，添加缓存象,把sql文件中的cache标签的所有信息否封装成cache类，然后保存到configuration中
    cacheElement(context.evalNode("cache"));
    // 创建 ParameterMapping 对象，封装成ParameterMap对象保存到configuration中，里面有namespace+id作为唯一id,还是有parameterClass对应到参数类型，然后对象到所有的配置的参数
    parameterMapElement(context.evalNodes("/mapper/parameterMap"));
    // 创建 List<ResultMapping>，与ParameterMapping基本类似，只是用的list,上面用的map
    resultMapElements(context.evalNodes("/mapper/resultMap"));
    // 解析可以复用的SQL,也是namespace+id作为唯一id，sql作为值保存到sqlFragments中
    sqlElement(context.evalNodes("/mapper/sql"));
    // 解析增删改查标签，得到 MappedStatement >>

    buildStatementFromContext(context.evalNodes("select|insert|update|delete"));
```

```
  } catch (Exception e) {
    throw new BuilderException("Error parsing Mapper XML. The XML location is '" +
resource + "'. Cause: " + e, e);
  }
}
```

- 重点分析buildStatementFromContext方法

```
private void buildStatementFromContext(List<XNode> list, String requiredDatabaseId) {
  // 因为一个文件里面有很多的sql语句，所以这里是list
  for (XNode context : list) {
    // 用来解析增删改查标签的 XMLStatementBuilder
    final XMLStatementBuilder statementParser = new
XMLStatementBuilder(configuration, builderAssistant, context, requiredDatabaseId);
    try {
      // 解析 Statement，添加 MappedStatement 对象 ，点进去
      statementParser.parseStatementNode();
    } catch (IncompleteElementException e) {
      configuration.addIncompleteStatement(statementParser);
    }
  }
}
```

- 分析parseStatementNode方法

```
public void parseStatementNode() {
    // sql的id
    String id = context.getStringAttribute("id");
    // 是否配置多数据源，一般都没有
    String databaseId = context.getStringAttribute("databaseId");
    // 如果配置了，但是和当前的数据源不匹配，则不解析
    if (!databaseIdMatchesCurrent(id, databaseId, this.requiredDatabaseId)) {
      return;
    }
    // 获取节点的名称
    String nodeName = context.getNode().getNodeName();
    // 获取sql的类型，select，update，delete，insert
    SqlCommandType sqlCommandType =
SqlCommandType.valueOf(nodeName.toUpperCase(Locale.ENGLISH));
    boolean isSelect = sqlCommandType == SqlCommandType.SELECT;
    boolean flushCache = context.getBooleanAttribute("flushCache", !isSelect);
    boolean useCache = context.getBooleanAttribute("useCache", isSelect);
    boolean resultOrdered = context.getBooleanAttribute("resultOrdered", false);

    // Include Fragments before parsing
    XMLIncludeTransformer includeParser = new XMLIncludeTransformer(configuration,
builderAssistant);
    includeParser.applyIncludes(context.getNode());
```

```
    String parameterType = context.getStringAttribute("parameterType");
    Class<?> parameterTypeClass = resolveClass(parameterType);

    String lang = context.getStringAttribute("lang");
    LanguageDriver langDriver = getLanguageDriver(lang);

    // Parse selectKey after includes and remove them.
    processSelectKeyNodes(id, parameterTypeClass, langDriver);


    KeyGenerator keyGenerator;
   // keyStatementId=sqlId!selectkey
    String keyStatementId = id + SelectKeyGenerator.SELECT_KEY_SUFFIX;
    keyStatementId = builderAssistant.applyCurrentNamespace(keyStatementId, true);
    if (configuration.hasKeyGenerator(keyStatementId)) {
      keyGenerator = configuration.getKeyGenerator(keyStatementId);
    } else {
      keyGenerator = context.getBooleanAttribute("useGeneratedKeys",
          configuration.isUseGeneratedKeys() &&
SqlCommandType.INSERT.equals(sqlCommandType))
          ? Jdbc3KeyGenerator.INSTANCE : NoKeyGenerator.INSTANCE;
    }

    SqlSource sqlSource = langDriver.createSqlSource(configuration, context,
parameterTypeClass);
    StatementType statementType =
StatementType.valueOf(context.getStringAttribute("statementType",
StatementType.PREPARED.toString()));
    Integer fetchSize = context.getIntAttribute("fetchSize");
    Integer timeout = context.getIntAttribute("timeout");
    String parameterMap = context.getStringAttribute("parameterMap");
    String resultType = context.getStringAttribute("resultType");
    Class<?> resultTypeClass = resolveClass(resultType);
    String resultMap = context.getStringAttribute("resultMap");
    String resultSetType = context.getStringAttribute("resultSetType");
    ResultSetType resultSetTypeEnum = resolveResultSetType(resultSetType);
    if (resultSetTypeEnum == null) {
      resultSetTypeEnum = configuration.getDefaultResultSetType();
    }
    String keyProperty = context.getStringAttribute("keyProperty");
    String keyColumn = context.getStringAttribute("keyColumn");
    String resultSets = context.getStringAttribute("resultSets");

    // >> 关键的一步： MappedStatement 的创建
    builderAssistant.addMappedStatement(id, sqlSource, statementType, sqlCommandType,
        fetchSize, timeout, parameterMap, parameterTypeClass, resultMap,
resultTypeClass,
        resultSetTypeEnum, flushCache, useCache, resultOrdered,
        keyGenerator, keyProperty, keyColumn, databaseId, langDriver, resultSets);
  }
```

- addMappedStatement方法分析

```
public MappedStatement addMappedStatement(
      String id,
      SqlSource sqlSource,
      StatementType statementType,
      SqlCommandType sqlCommandType,
      Integer fetchSize,
      Integer timeout,
      String parameterMap,
      Class<?> parameterType,
      String resultMap,
      Class<?> resultType,
      ResultSetType resultSetType,
      boolean flushCache,
      boolean useCache,
      boolean resultOrdered,
      KeyGenerator keyGenerator,
      String keyProperty,
      String keyColumn,
      String databaseId,
      LanguageDriver lang,
      String resultSets) {

    if (unresolvedCacheRef) {
      throw new IncompleteElementException("Cache-ref not yet resolved");
    }
    // 点击去，这个地方就是拼接namespace+sqlid的地方，保证sql的唯一性
    id = applyCurrentNamespace(id, false);
    // 如果是select，则isSelect=true
    boolean isSelect = sqlCommandType == SqlCommandType.SELECT;
    // 每一个sql都是一个statement
    MappedStatement.Builder statementBuilder = new
MappedStatement.Builder(configuration, id, sqlSource, sqlCommandType)
        .resource(resource)// sql的xml文件
        .fetchSize(fetchSize)
        .timeout(timeout)
        .statementType(statementType)// PREPARED
        .keyGenerator(keyGenerator)
        .keyProperty(keyProperty)
        .keyColumn(keyColumn)
        .databaseId(databaseId)
        .lang(lang)
        .resultOrdered(resultOrdered)// 结果排序，默认false
        .resultSets(resultSets)
        .resultMaps(getStatementResultMaps(resultMap, resultType, id))// 把返回类型
ResultMap保存到map中
        .resultSetType(resultSetType)
        .flushCacheRequired(valueOrDefault(flushCache, !isSelect))// 都为false,所以返回
false,是否刷新缓存，很明显select不用，其他的需要
        .useCache(valueOrDefault(useCache, isSelect))// 都为ture,使用缓存，如果不配置他，
默认使用缓存，如果使用它，根据他是false还是true

        .cache(currentCache);
```

```
    ParameterMap statementParameterMap = getStatementParameterMap(parameterMap,
parameterType, id);
    if (statementParameterMap != null) {
      statementBuilder.parameterMap(statementParameterMap);
    }

    MappedStatement statement = statementBuilder.build();
    // 最关键的一步，在 Configuration 添加了 MappedStatement >>
    configuration.addMappedStatement(statement);
    return statement;
  }
```

- 回到当初XMLMapperBuilder类中的parse方法中

```
public void parse() {
    // 总体上做了两件事情，对于语句的注册和接口的注册
    if (!configuration.isResourceLoaded(resource)) {
     // 到此，这些就全部解析完成
      configurationElement(parser.evalNode("/mapper"));
      configuration.addLoadedResource(resource);
      // 2、把namespace（接口类型）和工厂类绑定起来，放到一个map。
      // 一个namespace 一个 MapperProxyFactory，点进去
      bindMapperForNamespace();
    }

    parsePendingResultMaps();
    parsePendingCacheRefs();
    parsePendingStatements();
  }
```

- 分析方法bindMapperForNamespace

```
private void bindMapperForNamespace() {
    // 首先获取sql的xml文件的namespace
    String namespace = builderAssistant.getCurrentNamespace();
    if (namespace != null) {
      Class<?> boundType = null;
      try {
        // 得到namespace的类对象
        boundType = Resources.classForName(namespace);
      } catch (ClassNotFoundException e) {
        //ignore, bound type is not required
      }
      if (boundType != null) {
        if (!configuration.hasMapper(boundType)) {
          // 把"namespace:" + namespace放入到configuration中的一个set中进行保存
          configuration.addLoadedResource("namespace:" + namespace);

          // 添加到 MapperRegistry，本质是一个 map，里面也有 Configuration
```

```
            configuration.addMapper(boundType);
          }
        }
      }
    }
```

- 分析addMapper方法

```
public <T> void addMapper(Class<T> type) {
  // 判断是否是接口
  if (type.isInterface()) {
    if (hasMapper(type)) {
      throw new BindingException("Type " + type + " is already known to the
MapperRegistry.");
    }
    boolean loadCompleted = false;
    try {
      // ! Map<Class<?>, MapperProxyFactory<?>> 存放的是接口类型，和对应的工厂类的关系
      // 使用接口的class类型作为key,值为new MapperProxyFactory<>(type)工厂，后面可以根据这个
工厂获取到这个class类的代理类
      // 这里就是代理mapper接口的地方
      knownMappers.put(type, new MapperProxyFactory<>(type));
      // 注册了接口之后，根据接口，开始解析所有方法上的注解，例如 @Select >>
      MapperAnnotationBuilder parser = new MapperAnnotationBuilder(config, type);
      parser.parse();
      loadCompleted = true;
    } finally {
      if (!loadCompleted) {
        knownMappers.remove(type);
      }
    }
  }
}
```

到此全部解析完成

创建会话

- openSessionFromDataSource方法解析

```
private SqlSession openSessionFromDataSource(ExecutorType execType,
TransactionIsolationLevel level, boolean autoCommit) {
    Transaction tx = null;
    try {
      // 得到environment标签定义的所有信息
      final Environment environment = configuration.getEnvironment();
      // 获取事务工厂
      final TransactionFactory transactionFactory =
getTransactionFactoryFromEnvironment(environment);

      // 创建事务
```

```
      tx = transactionFactory.newTransaction(environment.getDataSource(), level,
autoCommit);
      // 根据事务工厂和默认的执行器类型，创建执行器 >>
      final Executor executor = configuration.newExecutor(tx, execType);
      return new DefaultSqlSession(configuration, executor, autoCommit);
    } catch (Exception e) {
      closeTransaction(tx); // may have fetched a connection so lets call close()
      throw ExceptionFactory.wrapException("Error opening session.  Cause: " + e, e);
    } finally {
      ErrorContext.instance().reset();
    }
  }
```

- 分析newExecutor方法

```
public Executor newExecutor(Transaction transaction, ExecutorType executorType) {
    // executorType默认情况下，是SIMPLE
    executorType = executorType == null ? defaultExecutorType : executorType;
    executorType = executorType == null ? ExecutorType.SIMPLE : executorType;
    Executor executor;
    // 这个是批量执行器
    if (ExecutorType.BATCH == executorType) {
      executor = new BatchExecutor(this, transaction);
      // 这个是REUSE执行器
    } else if (ExecutorType.REUSE == executorType) {
      executor = new ReuseExecutor(this, transaction);
    } else {
      // 默认 SimpleExecutor
      executor = new SimpleExecutor(this, transaction);
    }
    // 二级缓存开关，settings 中的 cacheEnabled 默认是 true
    if (cacheEnabled) {
      executor = new CachingExecutor(executor);
    }
    // 植入插件的逻辑，当初interceptorChain有两个自定义的插件
    executor = (Executor) interceptorChain.pluginAll(executor);
    return executor;
  }
```

- 分析pluginAll方法

```
 public Object pluginAll(Object target) {
    // 循环遍历，把执行器target传入进来，给执行器生成代理
    for (Interceptor interceptor : interceptors) {
      target = interceptor.plugin(target);
    }
    return target;
  }
```

```
public static Object wrap(Object target, Interceptor interceptor) {
   // 判断定义的插件上面的注解信息，查看拦截的是哪个对象，比如interface
org.apache.ibatis.executor.Executor
    Map<Class<?>, Set<Method>> signatureMap = getSignatureMap(interceptor);
  // 得到缓存执行器，如果没有开启缓存，就是简单执行器
    Class<?> type = target.getClass();
  // 里面会判断，是否拦截的是四大对象中的那个对象，从signatureMap中去判断
  // interfaces=interface org.apache.ibatis.executor.Executor
    Class<?>[] interfaces = getAllInterfaces(type, signatureMap);
    if (interfaces.length > 0) {
      // 此处大于0，那么就会生成代理对象
      return Proxy.newProxyInstance(
          type.getClassLoader(),
          interfaces,
          new Plugin(target, interceptor, signatureMap));
    }
    return target;
  }
```

- 重点分析new Plugin(target, interceptor, signatureMap),将来会调用Plugin的invoke方法

```
 @Override
  public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
    try {
      // 首先得到插件类上配置的是哪个执行器的哪个方法
      Set<Method> methods = signatureMap.get(method.getDeclaringClass());
      // 如果拦截是需要拦截的方法
      if (methods != null && methods.contains(method)) {
        // 则调用到拦截器的里面去
        return interceptor.intercept(new Invocation(target, method, args));
      }
      return method.invoke(target, args);
    } catch (Exception e) {
      throw ExceptionUtil.unwrapThrowable(e);
    }
  }
```

语句执行

- getMapper分析

```
public <T> T getMapper(Class<T> type) {
  // 获取到代理类
  return configuration.getMapper(type, this);
}
```

```
public <T> T getMapper(Class<T> type, SqlSession sqlSession) {
  // 当初放入到knownMappers中，根据类的类型去获取代理的工厂
  final MapperProxyFactory<T> mapperProxyFactory = (MapperProxyFactory<T>)
knownMappers.get(type);
  if (mapperProxyFactory == null) {
    throw new BindingException("Type " + type + " is not known to the
MapperRegistry.");
  }
  try {
    // 通过工厂得到mapper对象的代理类
    return mapperProxyFactory.newInstance(sqlSession);
  } catch (Exception e) {
    throw new BindingException("Error getting mapper instance. Cause: " + e, e);
  }
}
```

- 会去调用MapperProxy中的invoke方法

```
public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
  try {
    // toString hashCode equals getClass等方法，无需走到执行SQL的流程
    if (Object.class.equals(method.getDeclaringClass())) {
      return method.invoke(this, args);
    } else {
      // 提升获取 mapperMethod 的效率，到 MapperMethodInvoker（内部接口） 的 invoke
      // 普通方法会走到 PlainMethodInvoker（内部类） 的 invoke
      return cachedInvoker(method).invoke(proxy, method, args, sqlSession);
    }
  } catch (Throwable t) {
    throw ExceptionUtil.unwrapThrowable(t);
  }
}
```

- 然后会进入到MapperProx的内部类中的PlainMethodInvoker类中的invoke方法

```
@Override
public Object invoke(Object proxy, Method method, Object[] args, SqlSession
sqlSession) throws Throwable {
  // SQL执行的真正起点
  return mapperMethod.execute(sqlSession, args);
}
```

- 开始分析execute方法

```
public Object execute(SqlSession sqlSession, Object[] args) {
    Object result;
  // 判断是什么类型的查询
```

```
    switch (command.getType()) {
      case INSERT: {
        Object param = method.convertArgsToSqlCommandParam(args);
        result = rowCountResult(sqlSession.insert(command.getName(), param));
        break;
      }
      case UPDATE: {
        Object param = method.convertArgsToSqlCommandParam(args);
        result = rowCountResult(sqlSession.update(command.getName(), param));
        break;
      }
      case DELETE: {
        Object param = method.convertArgsToSqlCommandParam(args);
        result = rowCountResult(sqlSession.delete(command.getName(), param));
        break;
      }
      case SELECT:
        if (method.returnsVoid() && method.hasResultHandler()) {
          executeWithResultHandler(sqlSession, args);
          result = null;
          // 以这个作为例子，返回多条数据
        } else if (method.returnsMany()) {
          result = executeForMany(sqlSession, args);
        } else if (method.returnsMap()) {
          result = executeForMap(sqlSession, args);
        } else if (method.returnsCursor()) {
          result = executeForCursor(sqlSession, args);
        } else {
          Object param = method.convertArgsToSqlCommandParam(args);
          // 普通 select 语句的执行入口 >>
          result = sqlSession.selectOne(command.getName(), param);
          if (method.returnsOptional()
              && (result == null ||
!method.getReturnType().equals(result.getClass()))) {
            result = Optional.ofNullable(result);
          }
        }
        break;
      case FLUSH:
        result = sqlSession.flushStatements();
        break;
      default:
        throw new BindingException("Unknown execution method for: " +
command.getName());
    }
    if (result == null && method.getReturnType().isPrimitive() &&
!method.returnsVoid()) {
      throw new BindingException("Mapper method '" + command.getName()
          + " attempted to return null from a method with a primitive return type ("
+ method.getReturnType() + ").");
    }
    return result;

  }
```

- 方法分析executeForMany

```
private <E> Object executeForMany(SqlSession sqlSession, Object[] args) {
  List<E> result;
  // sql语句的入参
  Object param = method.convertArgsToSqlCommandParam(args);
  if (method.hasRowBounds()) {
    RowBounds rowBounds = method.extractRowBounds(args);
    result = sqlSession.selectList(command.getName(), param, rowBounds);
  } else {
    // 此处去拿到statement , command.getName()为namespace+id
    result = sqlSession.selectList(command.getName(), param);
  }
  // issue #510 Collections & arrays support
  if (!method.getReturnType().isAssignableFrom(result.getClass())) {
    if (method.getReturnType().isArray()) {
      return convertToArray(result);
    } else {
      return convertToDeclaredCollection(sqlSession.getConfiguration(), result);
    }
  }
  return result;
}
```

- 然后调用到selectList

```
public <E> List<E> selectList(String statement, Object parameter, RowBounds
rowBounds) {
  try {
    MappedStatement ms = configuration.getMappedStatement(statement);
    // 如果 cacheEnabled = true (默认) , Executor会被 CachingExecutor装饰,由于有插件, 所以此
处会生成一个代理对象, 会调用当初的plugin类中的invoke方法
    return executor.query(ms, wrapCollection(parameter), rowBounds,
Executor.NO_RESULT_HANDLER);
  } catch (Exception e) {
    throw ExceptionFactory.wrapException("Error querying database.  Cause: " + e, e);
  } finally {
    ErrorContext.instance().reset();
  }
}
```

- 调用完插件方法后，回到目标方法，正式去开始查询sql

```
public <E> List<E> query(MappedStatement ms, Object parameterObject, RowBounds
rowBounds, ResultHandler resultHandler) throws SQLException {
    // 获取SQL
    BoundSql boundSql = ms.getBoundSql(parameterObject);
    // 创建CacheKey: 什么样的SQL是同一条SQL?  >>
    CacheKey key = createCacheKey(ms, parameterObject, rowBounds, boundSql);
    return query(ms, parameterObject, rowBounds, resultHandler, key, boundSql);
  }
```

- 进入query方法

```
public <E> List<E> query(MappedStatement ms, Object parameterObject, RowBounds
rowBounds, ResultHandler resultHandler, CacheKey key, BoundSql boundSql)
      throws SQLException {
    Cache cache = ms.getCache();
    // cache 对象是在哪里创建的?  XMLMapperBuilder类 xmlconfigurationElement()
    // 由 <cache> 标签决定
    if (cache != null) {
      // flushCache="true" 清空一级二级缓存 >>
      flushCacheIfRequired(ms);
      // 如果使用缓存
      if (ms.isUseCache() && resultHandler == null) {
        ensureNoOutParams(ms, boundSql);
        // 获取二级缓存
        // 缓存通过 TransactionalCacheManager、TransactionalCache 管理
        @SuppressWarnings("unchecked")
        List<E> list = (List<E>) tcm.getObject(cache, key);
        if (list == null) {
          // 此处会调用BaseExecutor中的query方法
          list = delegate.query(ms, parameterObject, rowBounds, resultHandler, key,
boundSql);
          // 写入二级缓存
          tcm.putObject(cache, key, list); // issue #578 and #116
        }
        return list;
      }
    }
    // 走到 SimpleExecutor | ReuseExecutor | BatchExecutor
    return delegate.query(ms, parameterObject, rowBounds, resultHandler, key,
boundSql);
  }
```

- 进入baseExecutor中的query方法

```
public <E> List<E> query(MappedStatement ms, Object parameter, RowBounds rowBounds,
ResultHandler resultHandler, CacheKey key, BoundSql boundSql) throws SQLException {
    // 异常体系之 ErrorContext

    ErrorContext.instance().resource(ms.getResource()).activity("executing a
```

```
query").object(ms.getId());
    if (closed) {
      throw new ExecutorException("Executor was closed.");
    }
    if (queryStack == 0 && ms.isFlushCacheRequired()) {
      // flushCache="true"时，即使是查询，也清空一级缓存
      clearLocalCache();
    }
    List<E> list;
    try {
      // 防止递归查询重复处理缓存
      queryStack++;
      // 查询一级缓存
      // ResultHandler 和 ResultSetHandler的区别
      list = resultHandler == null ? (List<E>) localCache.getObject(key) : null;
      if (list != null) {
        handleLocallyCachedOutputParameters(ms, key, parameter, boundSql);
      } else {
        // 真正的查询流程
        list = queryFromDatabase(ms, parameter, rowBounds, resultHandler, key,
boundSql);
      }
    } finally {
      queryStack--;
    }
    if (queryStack == 0) {
      for (DeferredLoad deferredLoad : deferredLoads) {
        deferredLoad.load();
      }
      // issue #601
      deferredLoads.clear();
      if (configuration.getLocalCacheScope() == LocalCacheScope.STATEMENT) {
        // issue #482
        clearLocalCache();
      }
    }
    return list;
  }
```

- 下面正式发送sql语句去查询,queryFromDatabase方法分析

```
private <E> List<E> queryFromDatabase(MappedStatement ms, Object parameter, RowBounds
rowBounds, ResultHandler resultHandler, CacheKey key, BoundSql boundSql) throws
SQLException {
    List<E> list;
    // 先占位
    localCache.putObject(key, EXECUTION_PLACEHOLDER);
    try {
      // 三种 Executor 的区别，看doUpdate
      // 默认Simple

      list = doQuery(ms, parameter, rowBounds, resultHandler, boundSql);
```

```
    } finally {
      // 移除占位符
      localCache.removeObject(key);
    }
    // 写入一级缓存
    localCache.putObject(key, list);
    if (ms.getStatementType() == StatementType.CALLABLE) {
      localOutputParameterCache.putObject(key, parameter);
    }
    return list;
  }
```

- 通过doQuery方法会进入到SimpleExecutor类中

```
@Override
  public <E> List<E> doQuery(MappedStatement ms, Object parameter, RowBounds
rowBounds, ResultHandler resultHandler, BoundSql boundSql) throws SQLException {
    Statement stmt = null;
    try {
      Configuration configuration = ms.getConfiguration();
      // 此处返回的是一个代理对象，这个代理对象针对插件类中拦截StatementHandler类中的方法做处理
      // 并且里面还做了几件事情
      StatementHandler handler = configuration.newStatementHandler(wrapper, ms,
parameter, rowBounds, resultHandler, boundSql);
      // 获取一个 Statement对象,此处会去调用拦截了StatementHandler对象的setParameters方法
      stmt = prepareStatement(handler, ms.getStatementLog());
      // 执行查询
      return handler.query(stmt, resultHandler);
    } finally {
      // 用完就关闭
      closeStatement(stmt);
    }
  }
```

- configuration.newStatementHandler解析，比较重要

```
 public StatementHandler newStatementHandler(Executor executor, MappedStatement
mappedStatement, Object parameterObject, RowBounds rowBounds, ResultHandler
resultHandler, BoundSql boundSql) {
   // 首先初始化RoutingStatementHandler，但是在这个里面可以new一个
    StatementHandler statementHandler = new RoutingStatementHandler(executor,
mappedStatement, parameterObject, rowBounds, resultHandler, boundSql);
    // 植入插件逻辑（返回代理对象）
    statementHandler = (StatementHandler)
interceptorChain.pluginAll(statementHandler);
    return statementHandler;
  }
```

```
public RoutingStatementHandler(Executor executor, MappedStatement ms, Object
```

```
parameter, RowBounds rowBounds, ResultHandler resultHandler, BoundSql boundSql) {
    // StatementType 是怎么来的？ 增删改查标签中的 statementType="PREPARED", 默认值
PREPARED
    switch (ms.getStatementType()) {
      case STATEMENT:
        delegate = new SimpleStatementHandler(executor, ms, parameter, rowBounds,
resultHandler, boundSql);
        break;
      case PREPARED:
        // 创建 StatementHandler 的时候做了什么
        delegate = new PreparedStatementHandler(executor, ms, parameter, rowBounds,
resultHandler, boundSql);
        break;
      case CALLABLE:
        delegate = new CallableStatementHandler(executor, ms, parameter, rowBounds,
resultHandler, boundSql);
        break;
      default:
        throw new ExecutorException("Unknown statement type: " +
ms.getStatementType());
    }

  }
```

- 然后在初始化PreparedStatementHandler中，对拦截器拦截了parameterHandler类以及resultSetHandler类保存起来并生成代理类

```
protected BaseStatementHandler(Executor executor, MappedStatement mappedStatement,
Object parameterObject, RowBounds rowBounds, ResultHandler resultHandler, BoundSql
boundSql) {
    this.configuration = mappedStatement.getConfiguration();
    this.executor = executor;
    this.mappedStatement = mappedStatement;
    this.rowBounds = rowBounds;

    this.typeHandlerRegistry = configuration.getTypeHandlerRegistry();
    this.objectFactory = configuration.getObjectFactory();

    if (boundSql == null) { // issue #435, get the key before calculating the
statement
      generateKeys(parameterObject);
      boundSql = mappedStatement.getBoundSql(parameterObject);
    }

    this.boundSql = boundSql;

    // 创建了四大对象的其它两大对象 >>
    // 创建这两大对象的时候分别做了什么?
    // 拦截器拦截了parameterHandler类以及resultSetHandler类保存起来并生成代理类


    this.parameterHandler = configuration.newParameterHandler(mappedStatement,
```

```
parameterObject, boundSql);
    this.resultSetHandler = configuration.newResultSetHandler(executor,
mappedStatement, rowBounds, parameterHandler, resultHandler, boundSql);
  }
```

- 一当调用StatementHandler对象的setParameters方法进入到插件的逻辑中，plugin类中的invoke方法，很明显，现在是可以对入参做处理

```
  private Statement prepareStatement(StatementHandler handler, Log statementLog)
throws SQLException {
    Statement stmt;
    Connection connection = getConnection(statementLog);
    // 获取 Statement 对象
    stmt = handler.prepare(connection, transaction.getTimeout());
    // 为 Statement 设置参数，此处走插件逻辑
    handler.parameterize(stmt);
    return stmt;
  }
```

- 然后看到会在真正调用query方法后，通过resultHander进行结果集的拦截

```
@Override
public <E> List<E> query(Statement statement, ResultHandler resultHandler) throws
SQLException {
  PreparedStatement ps = (PreparedStatement) statement;
  // 到了JDBC的流程
  ps.execute();
  // 处理结果集
  return resultSetHandler.handleResultSets(ps);
}
```

Mybatis执行器的区别

确认我们的执行器SIMPLE, REUSE, BATCH

什么叫预编译处理？mysql对sql提前解析，并且可以防止sql注入

mybatis插件

1.Excutor:我要查看mybatis的sql执行过程

2.ParameterHandler我要修改他的参数

3.StatementHandler

4.ResultSetHandler我要拦截返回值

mybatis的一级缓存和二级缓存

一级缓存：它指的是Mybatis中sqlSession对象的缓存，当我们执行查询以后，查询的结果会同时存入到SqlSession为我们提供的一块区域中，该区域的结构是一个Map，当我们再次查询同样的数据，mybatis会 先去sqlsession中查询是否有，的话直接拿出来用，当SqlSession对象消失时，mybatis的一级缓存也就消失了，同时一级缓存是SqlSession范围的缓存，当调用SqlSession的修改、添加、删除、commit(),close等 方法时，就会清空一级缓存。

二级缓存：他只的是Mybatis中SqlSessionFactory对象的缓存，由同一个SqlSessionFactory对象创建的SqlSession共享其缓存，但是其中缓存的是数据而不是对象，所以从二级缓存再次查询出得结果的对象与 第一次存入的对象是不一样的。

例子：一级缓存

```
/**
    * 查询一级缓存
    * @throws IOException
    */
   @Test
   public void testOneCacheSelect() throws IOException {
       SqlSession session = sqlSessionFactory.openSession(); // ExecutorType.BATCH
       try {
           // 第一次获取该数据
           BlogMapper mapper = session.getMapper(BlogMapper.class);
           Blog blog1 = mapper.selectBlogById(1);
           System.out.println(blog1);
           // 第二次获取该数据
           Blog blog2 = mapper.selectBlogById(1);
           System.out.println(blog2);
           System.out.println(blog1==blog2);
       } finally {
           session.close();
       }
   }
```

清空一级缓存

```
/**
    * 清除一级缓存
    * @throws IOException
    */
   @Test
   public void testClearOneCacheSelect() throws IOException {
       SqlSession session = sqlSessionFactory.openSession(); // ExecutorType.BATCH
       try {
```

```
            // 第一次获取该数据
            BlogMapper mapper = session.getMapper(BlogMapper.class);
            Blog blog1 = mapper.selectBlogById(1);
            System.out.println(blog1);

//            session.commit();//调用SqlSession的commit方法清空缓存
            Blog blog = new Blog();
            blog.setBid(1);
            blog.setName("wentai");
            mapper.updateByPrimaryKey(blog);
            // 第二次获取该数据
            Blog blog2 = mapper.selectBlogById(1);
            System.out.println(blog2);
            System.out.println(blog1==blog2);
        } finally {
            session.close();
        }
    }
```

查询二级缓存

- 开启二级缓存

```
<setting name="cacheEnabled" value="true"/>
```

- xml文件中

```
<cache/>
```

- 然后在sql语句上添加标签 userCache

```
 <select id="selectBlogById" resultMap="BaseResultMap" useCache="true"
 statementType="PREPARED" >
        select * from blog where bid = #{bid}
    </select>
```

- 例子

```
/**
       * 二级缓存
       *
       * @throws IOException

       */
```

```java
@Test
public void testTwoCacheSelect() throws IOException {

  SqlSession session = sqlSessionFactory.openSession(); // ExecutorType.BATCH
  // //第一次查询 并更新二级缓存
  BlogMapper mapper = session.getMapper(BlogMapper.class);
  Blog blog1 = mapper.selectBlogById(1);
  System.out.println(blog1);
  session.commit();//commit()方法提交二级缓存 同时清空一级缓存
  session.close();

  //第二次查找命中二级缓存
  SqlSession session1 = sqlSessionFactory.openSession();
  BlogMapper mapper1 = session1.getMapper(BlogMapper.class);
  // 第二次获取该数据
  Blog blog2 = mapper1.selectBlogById(1);
  session1.commit();//commit()方法提交二级缓存 同时清空一级缓存
  session1.close();
  System.out.println(blog2);
  System.out.println(blog1 == blog2);

}
```

总结：mybatis的的一级缓存是SqlSession级别的缓存，一级缓存缓存的是对象，当SqlSession提交、关闭以及其他的更新数据库的操作发生后，一级缓存就会清空。二级缓存是SqlSessionFactory级别的缓存，同一个SqlSessionFactory产生的SqlSession都共享一个二级缓存，二级缓存中存储的是数据，当命中二级缓存时，通过存储的数据构造对象返回。查询数据的时候，查询的流程是二级缓存-一级缓存-数据库

Mybatis和spring集成